

PLAYWRIGHT PYTHON AUTOMATION CAPSTONE PROJECT & ASSIGNMENT

End-to-End Framework, API Automation, GitHub & CI/CD

Duration

5-7 Days

Evaluation

100 Marks

1. Project Scenario

You are part of a QA Automation team responsible for building a maintainable Playwright Python automation framework for a web application. The framework must support UI and API automation, reusable design, diagnostics, source control, continuous integration, reporting, artifacts and email notification.

Recommended public practice applications:

- UI: <https://www.saucedemo.com/>
- UI: <https://demoqa.com/>
- API: <https://jsonplaceholder.typicode.com/>

2. Business Requirement

The automation framework should provide repeatable validation without relying on manual execution and should support the following capabilities:

- Smoke and regression testing
- Read-only validation
- UI automation
- API automation
- Data-driven testing
- Parallel execution
- Failure diagnostics
- CI execution
- Automated reporting and notifications

3. Expected Framework Structure

```
playwright-capstone/  
├── pages/  
│   ├── login_page.py  
│   └── products_page.py
```

```
| └─ registration_page.py
├─ tests/
|   └─ ui/
|       └─ api/
├─ testdata/
|   └─ login_data.json
|       └─ registration_data.json
├─ utils/
|   └─ logger.py
|   └─ json_utils.py
|       └─ session_storage.py
├─ logs/
├─ screenshots/
├─ conftest.py
├─ pytest.ini
├─ requirements.txt
├─ .gitignore
└─ README.md
```

The framework must be modular. Test logic, page locators, data, reusable utilities and configuration should not be placed in a single file.

4. UI Automation Requirements

Automate a minimum of 8 UI test cases. Suggested scenarios include:

- Valid login and successful navigation
- Invalid login and error-message validation
- Verify products are displayed
- Add a product to cart and validate cart count
- Remove a product from cart
- Verify product details
- Sort products
- Complete checkout and validate successful order

Use semantic Playwright locators such as `get_by_role()`, `get_by_text()`, `get_by_label()`, `get_by_placeholder()` and `locator()`. Avoid unnecessary XPath where stable semantic locators are available.

5. Page Object Model (POM)

UI tests should focus on business scenarios. Page-specific locators and actions must be encapsulated in Page Object classes.

```
class LoginPage:
    def __init__(self, page):
        self.page = page
        self.username = page.locator("#user-name")
        self.password = page.locator("#password")
```

```
self.login_button = page.locator("#login-button")
```

```
def login(self, username, password):  
    self.username.fill(username)  
    self.password.fill(password)  
    self.login_button.click()
```

6. Data-Driven Testing

Keep test data outside the test files. At least one test must execute with 3 different JSON datasets using `json.load()` and `@pytest.mark.parametrize`.

```
[  
    {"username": "standard_user", "password": "secret_sauce"},  
    {"username": "locked_out_user", "password": "secret_sauce"},  
    {"username": "invalid_user", "password": "wrong_password"}  
]
```

7. Fixtures, Hooks and Markers

Create reusable fixtures in `conftest.py`. At least one fixture must use `yield` to demonstrate setup and teardown. Register and use the markers `smoke`, `regression` and `readonly`.

```
[pytest]  
markers =  
    smoke: critical smoke tests  
    regression: regression tests  
    readonly: tests that do not modify application data
```

Trainees must demonstrate execution commands such as:

```
pytest -m smoke  
pytest -m "smoke or regression"
```

8. Multiple Windows / Tabs

Create at least one test using `page.context.expect_page()`. Validate the number of open pages, parent URL, child URL and child-page content. Recommended practice URL: <https://the-internet.herokuapp.com/windows>

9. Wait Handling

Demonstrate Playwright auto-waiting and explicit condition-based waits. Include locator, URL and load-state waits. Do not use `time.sleep()` as the primary synchronization mechanism.

```
locator.wait_for(state="visible")  
page.wait_for_url("**/inventory.html")
```

```
page.wait_for_load_state("domcontentloaded")
```

10. API Automation

Create at least 5 API tests using Playwright `APIRequestContext`. Cover GET, POST, PUT or PATCH, and DELETE. Validate HTTP status codes, response bodies, individual JSON fields and list data.

```
response = request.get("/users")
assert response.status == 200
data = response.json()
assert data[0]["id"] == 1
```

11. API Response Model

Create at least one response model class that encapsulates response data and exposes fields using properties. Convert JSON responses into model objects and perform validations through the model properties.

```
class User:
    def __init__(self, id, name, username, email):
        self._id = id
        self._name = name
        self._username = username
        self._email = email

    @property
    def id(self):
        return self._id
```

12. Search Without Knowing the Index

Do not assume that the required object is always `users[0]`. Find an object using a business value such as name or id, then validate another property.

```
user = next(
    (u for u in users if u.name == "Leanne Graham"),
    None
)
assert user is not None
```

13. Deep JSON Comparison

Compare two JSON response variables. First demonstrate exact comparison using `assert actual == expected`. Then implement a reusable deep-comparison utility capable of reporting a mismatch path such as `root.address.city`.

14. Session Storage

Create reusable functions to write, read, save, clear, restore and validate sessionStorage. Save captured values into session_data.json. Trainees must explain why page.context.cookies() does not capture sessionStorage.

```
session_data = page.evaluate("""
() => Object.fromEntries(Object.entries(sessionStorage))
""")
```

15. Logging

Implement Python logging and create logs/automation.log. Log important execution milestones and failures using appropriate levels such as DEBUG, INFO, WARNING and ERROR. Never log passwords, tokens or other secrets.

16. Failure Artifacts

Configure Playwright artifacts for troubleshooting:

```
--tracing retain-on-failure
--screenshot only-on-failure
--video retain-on-failure
```

Trainees must be able to explain the difference between a screenshot, video and Playwright trace.

17. HTML Reporting

Generate a pytest HTML report using pytest-html:

```
pytest --html=report.html --self-contained-html
```

18. Git & GitHub Requirements

Each trainee or team must maintain the framework in a GitHub repository and demonstrate:

- Create repository
- Clone or initialize repository
- Use main/master as the integration branch
- Create at least one feature branch
- Commit and push changes
- Create a Pull Request
- Merge the Pull Request

Example branches: feature/api-tests, feature/login-automation

19. .gitignore

Generated or machine-specific files should not be committed unnecessarily.

```
venv/  
__pycache__/  
.pytest_cache/  
logs/  
screenshots/  
test-results/  
allure-results/  
*.log
```

20. GitHub Actions CI

Create `.github/workflows/playwright.yml`. The workflow must check out the repository, configure Python, install dependencies, install Playwright Chromium, execute pytest, generate the HTML report and upload artifacts.

```
- name: Checkout repository  
  uses: actions/checkout@v4  
  
- name: Set up Python  
  uses: actions/setup-python@v5  
  with:  
    python-version: "3.12"  
  
- name: Install dependencies  
  run: |  
    python -m pip install --upgrade pip  
    pip install -r requirements.txt  
  
- name: Install Playwright Chromium  
  run: |  
    playwright install --with-deps chromium  
  
- name: Run Tests  
  run: |  
    pytest --html=report.html --self-contained-html
```

21. Workflow Triggers

The workflow must support push, pull_request and workflow_dispatch triggers. Trainees must explain when each trigger runs.

```
on:  
  push:  
    branches: [main]  
  pull_request:  
    branches: [main]
```

```
workflow_dispatch:
```

22. Artifact Upload

```
- name: Upload HTML Report
  id: upload-report
  if: always()
  uses: actions/upload-artifact@v4
  with:
    name: pytest-html-report
    path: report.html
```

Playwright failure artifacts should also be uploaded when available.

23. Email Notification

After execution, send an email that contains execution status, repository/workflow information, artifact/report URL and the HTML report attachment. Credentials must be stored in GitHub Secrets and never hard-coded in YAML.

```
username: ${ secrets.EMAIL_USERNAME }
password: ${ secrets.EMAIL_PASSWORD }

Artifact URL:
${ steps.upload-report.outputs.artifact-url }
```

24. README Documentation

README.md must explain the project objective, framework architecture, prerequisites, installation, Playwright setup, execution commands, markers, parallel execution, report generation, CI workflow and project folder structure.

```
pip install -r requirements.txt
playwright install
pytest
pytest -m smoke
pytest -n 4
```

25. Final Deliverables

- GitHub repository URL
- Complete Playwright Python framework
- Minimum 8 UI tests
- Minimum 5 API tests
- Page Object Model implementation
- JSON data-driven implementation

- Fixtures and teardown
- Pytest markers
- Multiple-window test
- Session-storage capture and restore
- Logging implementation
- API response-model implementation
- Deep JSON comparison
- Screenshots/traces/videos on failure
- HTML report
- GitHub Actions workflow
- Successful CI execution run
- Uploaded HTML report artifact
- Email notification with report attached
- README documentation

26. Evaluation - 100 Marks

Evaluation Area	Marks
Framework structure and code quality	10
Page Object Model	10
UI automation	15
API automation	10
Data-driven testing	5
Fixtures, hooks and markers	10
Session storage / browser handling	5
Logging and exception handling	5
Reports, screenshots, trace and video	5
Git / branching / Pull Request	5
GitHub Actions CI	10
Artifact + email integration	5
README / documentation	5
Total	100

Suggested Grading Bands

- 90-100: Excellent
- 80-89: Very Good
- 70-79: Good
- 60-69: Needs Improvement
- Below 60: Rework Required

27. Mandatory Completion Rule

The capstone is not complete if the framework only works locally. The framework must execute successfully from GitHub Actions and produce a downloadable test report artifact. This ensures the assignment demonstrates a complete automation lifecycle rather than isolated Playwright scripts.

28. Suggested Submission Format

- Repository link
- README with setup and execution instructions
- GitHub Actions run link
- HTML report artifact
- Email notification screenshot
- Short 5-10 minute project walkthrough explaining architecture, design choices and one failure-debugging example